Chapter 15: Best Practices, Common Pitfalls, and Next Steps

Download this chapter as a PDF for offline reading:
Chapter-15-Best-Practices-Common-Pitfalls-and-Next-Steps.pdf

You now have the core skills: writing queries, using the language libraries, tracking data and taint, extending models, packaging work for production, and integrating it into CI. This final chapter collects the habits that keep that work effective and points to where to go next.

Effective Habits

Start narrow, then generalize

Write the smallest query that finds a known example. Only after it works should you relax sources, sinks, or structural conditions. This is the same seed-to-variant process from Chapter 14, applied every day.

Prefer library predicates

If the standard library already models a concept, use it. Hand-rolled AST walks are slower, more brittle, and harder for others to maintain.

Keep metadata complete

Every query that leaves your laptop should have `@name`, `@description`, `@kind`, `@id`, `@tags`, severity, and precision. Incomplete metadata makes the query harder to operate in code scanning and in CI.

Test on real code

A query that passes on a toy example can still be wrong. Run it against at least one substantial database (or with MRVA) before you trust it. Inspect a few paths, not just the alert count.

Model once, benefit everywhere

When you discover a missing source, sink, or summary, put it in a library pack (Chapter 10) rather than duplicating the logic inside a single query. Every future query then inherits the improvement.

Measure before optimizing

If a query is slow, time it and look for the usual causes (broad `exists`, missing early filters, accidental global flow). Do not guess. Chapter 13's techniques are usually enough.

Common pitfalls

- Using pure data flow when the transformation requires taint tracking.
- Forgetting a barrier and reporting sanitized flows.
- An `@id` that collides with another query.

- A suite that silently excludes the query you just wrote.
- Treating the first MRVA results as final without reviewing false positives.

Next Steps

Immediate

- Re-read any chapter whose exercises you skipped and complete them.
- Take one query you wrote while reading this book and put it through the production checklist in Chapter 11.
- Run it in a GitHub Actions workflow (Chapter 12) on a repository you control.

Short term

- Contribute a missing model or a small query improvement upstream, or publish it in your own pack.
- Explore the offline reference in the companion repository (`reference/`). It is structured for both human reading and for AI assistants.
- Try multi-repository variant analysis on a query you trust.

Longer term

- Follow the official CodeQL documentation and the Security Lab blog for new library models and language support.
- When a new vulnerability class appears, apply the seed-to-variant process instead of starting from scratch.
- Teach someone else. The fastest way to solidify the material is to explain a path query or a custom model to a colleague.

Appendix: Resources

- Official CodeQL documentation: https://codeql.github.com/docs/
- Standard libraries: https://codeql.github.com/codeql-standard-libraries/
- Companion repository (examples and reference): https://github.com/rod-trent/MustLearnCodeQL
- Chapter exercises: see `examples/chapterNN/EXERCISES.md` for Chapters 8–14

Closing

CodeQL's power comes from treating code as data you can query. You now have the tools to do that for exploration, for security, and for production automation. The queries you write next will be better than the ones you wrote at the start of the book—and the ones after those will be better still.

Thank you for reading *Must Learn CodeQL*.

All examples and the offline reference live in the companion repository: github.com/rod-trent/MustLearnCodeQL

Must Learn CodeQL — Rod Trent
Companion repository: github.com/rod-trent/MustLearnCodeQL